



# Setup & Run Guide

Real-Time Traffic & Transit Congestion Intelligence

Greater Cairo and Alexandria

---

Digital Egypt Pioneers Initiative (DEPI)

Data Engineering Track — Capstone Project

**Version 2.0 — July 2026**

Team Lead: Mohamed Mahmoud

# Table of Contents

1. Prerequisites
2. Clone and Configure
3. Start the Platform
4. Run the Spark Streaming Jobs
5. Verify Data Flow
6. Access the UIs
7. Useful Commands
8. Stop and Restart
9. Troubleshooting
10. Architecture Overview

# 1. Prerequisites

## Required Software

| Software       | Version | Download                                                                    |
|----------------|---------|-----------------------------------------------------------------------------|
| Docker Desktop | 20.10+  | <a href="https://docs.docker.com/get-docker">docs.docker.com/get-docker</a> |
| Docker Compose | v2.0+   | Included with Docker Desktop                                                |
| Git            | 2.30+   | <a href="https://git-scm.com/downloads">git-scm.com/downloads</a>           |

## System Requirements

- **RAM:** At least 8 GB allocated to Docker (recommended: 12 GB)
- **Disk:** At least 5 GB free for Docker images and volumes
- **OS:** Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)

**Important:** On 16 GB machines, close heavy apps (browsers with many tabs, Spotify, etc.) while the platform runs. The 18 Docker containers + 3 Spark JVMs can use 8-10 GB.

## API Keys

| API                | Required?     | Free Tier     | Signup                                                          |
|--------------------|---------------|---------------|-----------------------------------------------------------------|
| TomTom Routing API | Yes           | 2,500 req/day | <a href="https://developer.tomtom.com">developer.tomtom.com</a> |
| Discord Webhook    | Optional      | Unlimited     | Discord Server Settings                                         |
| Open-Meteo         | No key needed | Unlimited     | <a href="https://open-meteo.com">open-meteo.com</a>             |

## 2. Clone and Configure

### 2.1 Clone the Repository

```
git clone https://github.com/mohamed-mahmoud-de/NileFlow.git
cd NileFlow
```

### 2.2 Create Your Environment File

Copy the example and fill in your values:

#### Windows (PowerShell)

```
Copy-Item .env.example .env
```

#### macOS / Linux

```
cp .env.example .env
```

### 2.3 Edit the .env File

Open .env in any text editor and set your values:

```
# REQUIRED: Get your free key from developer.tomtom.com
TOMTOM_API_KEY=your_actual_api_key_here

# OPTIONAL: Discord webhook for real-time alerts
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...

# The rest can stay as defaults for local development
```

**Important:** Without a valid TOMTOM\_API\_KEY, the traffic producer will fail and congestion data will be empty.

## 3. Start the Platform

### 3.1 Start All Services

```
docker compose up -d
```

This starts **18 services**. On first run Docker pulls all images (~5 GB). Subsequent starts are much faster.

### 3.2 What Happens Automatically

1. Zookeeper and Kafka start (message broker)
2. PostgreSQL starts with schema SQL + corridor seed data
3. Cassandra starts; cassandra-init creates keyspace + tables
4. Redis starts (alert pub/sub + vehicle position cache)
5. Elasticsearch starts; elasticsearch-init creates alert indices
6. Airflow init migrates DB and creates admin user (admin / admin)
7. Airflow webserver and scheduler start
8. Spark master and worker start (6 cores, 3 GB)
9. 3 Kafka producers start streaming data immediately
10. Streamlit dashboard starts on port 8501
11. Discord alerter subscribes to Redis Pub/Sub

### 3.3 Wait for Initialization

Some services take time to initialize (especially Cassandra and Elasticsearch). **Wait about 2-3 minutes** for everything to be ready.

Monitor startup progress:

```
docker compose logs -f cassandra-init elasticsearch-init
```

### 3.4 Verify Containers

```
docker compose ps
```

All services should show **Up** status except cassandra-init, elasticsearch-init, and airflow-init which exit with code 0 after completing initialization.

## 4. Run the Spark Streaming Jobs

**Important:** This is the most important step! The dashboard reads from Cassandra, and **only the Spark jobs write there**. If you skip this step, all KPIs and charts will be empty.

### 4.1 Submit All 3 Jobs

#### Windows (PowerShell)

```
.\scripts\run_spark_jobs.ps1
```

#### macOS / Linux / Git Bash

```
./scripts/run_spark_jobs.sh
```

The script submits three detached Spark streaming applications:

| Job               | Kafka Topic             | Cassandra Table       | Window             |
|-------------------|-------------------------|-----------------------|--------------------|
| Traffic Processor | traffic_events          | congestion_metrics    | 10 min             |
| Weather Processor | weather_events          | weather_metrics       | N/A (pass-through) |
| Vehicle Processor | vehicle_position_events | vehicle_speed_metrics | 5 min              |

**Note:** On first run, Spark downloads connector JARs (~1-2 minutes). They are cached afterwards in the `spark-ivy-cache` volume.

### 4.2 Verify Jobs Are Running

Open the Spark master UI at <http://localhost:8081>. You should see **3 Running Applications**:

- NileFlow-TrafficProcessor
- NileFlow-WeatherProcessor
- NileFlow-VehiclePositionsProcessor

Check job logs for errors:

#### Windows

```
Get-Content spark\logs\traffic_processor.log -Tail 20 -Wait
```

#### macOS / Linux

```
tail -f spark/logs/traffic_processor.log
```

### 4.3 Stop the Jobs

### Windows (PowerShell)

```
.\scripts\stop_spark_jobs.ps1
```

### macOS / Linux / Git Bash

```
./scripts/stop_spark_jobs.sh
```

**Important:** Do NOT submit the jobs with raw `spark-submit` commands — the scripts set `--conf spark.cores.max=2` per job. Without this, the first job grabs all worker cores and the other two hang forever in `WAITING` state.

## 5. Verify Data Flow

### 5.1 Check Producers

```
docker logs nileflow-traffic-producer --tail 5
docker logs nileflow-weather-producer --tail 5
docker logs nileflow-vehicle-positions-producer --tail 5
```

### 5.2 Check Cassandra Tables

Row counts should grow every few minutes:

```
docker exec nileflow-cassandra cqlsh -e "SELECT COUNT(*) FROM
nileflow.congestion_metrics;"
docker exec nileflow-cassandra cqlsh -e "SELECT COUNT(*) FROM
nileflow.weather_metrics;"
docker exec nileflow-cassandra cqlsh -e "SELECT COUNT(*) FROM
nileflow.vehicle_speed_metrics;"
```

### 5.3 Check Elasticsearch

#### macOS / Linux / Git Bash

```
curl http://localhost:9200/_cat/indices?v
```

#### Windows (PowerShell)

```
Invoke-WebRequest -UseBasicParsing "http://localhost:9200/_cat/indices?v" |
Select-Object -ExpandProperty Content
```

## Timing Expectations

| Table                 | First Data After | Reason                                        |
|-----------------------|------------------|-----------------------------------------------|
| weather_metrics       | ~1 min           | Weather publishes every 5 min                 |
| congestion_metrics    | ~2-3 min         | Traffic publishes every 2 min, 10-min windows |
| vehicle_speed_metrics | ~1-2 min         | Positions every 10s, 5-min windows            |

## 6. Access the UIs

| UI                    | URL                   | Login         |
|-----------------------|-----------------------|---------------|
| Dashboard (Streamlit) | http://localhost:8501 | None          |
| Airflow               | http://localhost:8080 | admin / admin |
| Spark Master          | http://localhost:8081 | None          |
| Elasticsearch         | http://localhost:9200 | None          |

### Dashboard Features

- **Header metrics:** corridors monitored, avg congestion, avg speed, anomalies, vehicles
- **Live congestion map:** color-coded corridors (green = free flow, red = gridlock) with vehicle markers
- **Corridor status table:** real-time congestion level and speed per corridor
- **Time-series charts:** congestion index and speed over the last 2 hours
- **Alert feed:** latest anomaly alerts from Elasticsearch
- **Weather sidebar:** current conditions for Cairo and Alexandria

The dashboard auto-refreshes every 10 seconds.

### Airflow DAGs

| DAG                         | Schedule     | Purpose                                  |
|-----------------------------|--------------|------------------------------------------|
| nileflow_daily_data_refresh | @daily       | Upsert corridors + recalculate baselines |
| nileflow_data_quality       | Every 30 min | Kafka / Cassandra / ES freshness checks  |

## 7. Useful Commands

### View Logs

```
docker compose logs -f # All services
docker compose logs -f traffic-producer # Specific service
```

### Query Cassandra

```
docker exec -it nileflow-cassandra cqlsh
USE nileflow;
SELECT * FROM congestion_metrics LIMIT 10;
SELECT * FROM weather_metrics LIMIT 10;
SELECT * FROM vehicle_speed_metrics LIMIT 10;
```

### Query PostgreSQL

```
docker exec -it nileflow-postgres psql -U nileflow -d nileflow
SET search_path TO nileflow;
SELECT * FROM corridors;
SELECT * FROM congestion_baselines LIMIT 10;
```

### Query Elasticsearch

#### macOS / Linux / Git Bash

```
curl http://localhost:9200/_cluster/health?pretty
curl "http://localhost:9200/congestion_alerts/_search?pretty&size=5"
```

#### Windows (PowerShell)

```
Invoke-WebRequest -UseBasicParsing "http://localhost:9200/_cluster/health?pretty" |
Select-Object -ExpandProperty Content
```

### Redis

```
docker exec -it nileflow-redis redis-cli
GET nileflow:vehicle_positions
SUBSCRIBE nileflow:alerts
```

## 8. Stop and Restart

```
docker compose down # Stop all (keeps data volumes)
docker compose down -v # Stop + delete all data
docker compose restart dashboard # Restart a specific service
```

**Important:** After a full restart (`docker compose up -d`), you must re-run the Spark jobs script (Step 4). The Spark streaming jobs do NOT auto-start with the stack.

## 9. Troubleshooting

### KPIs / charts are empty

The Spark jobs aren't running. Check `http://localhost:8081` — you need 3 Running Applications. Re-run the Spark jobs script from Step 4. Then check `spark/logs/*.log` for errors.

---

### A Spark job is stuck in WAITING

Not enough free worker cores. Stop all jobs with the stop script, then re-submit with the run script. The scripts cap each job at 2 cores so all three fit on the 6-core worker.

---

### Dashboard shows 'NoHostAvailable'

The Cassandra session cache is stale. Just restart the dashboard: `docker compose restart dashboard`. The updated code auto-recovers on the next refresh.

---

### Weather timestamps are ~3h ahead

Old bug — the producer used to request Africa/Cairo timezone from Open-Meteo. It now requests UTC. Restart the weather producer: `docker compose restart weather-producer`.

---

### cqlsh times out on DDL or COUNT

Cassandra's small heap makes it slow. Add `--request-timeout=120` to the cqlsh command.

---

### Traffic producer keeps restarting

Almost always a missing or invalid `TOMTOM_API_KEY` in `.env`. Check: `docker logs nileflow-traffic-producer --tail 20`.

---

### Docker Desktop unresponsive / 500 errors

Out of memory. This platform uses ~8-10 GB RAM. Close heavy applications, or increase Docker memory allocation: Docker Desktop > Settings > Resources > Memory > 10+ GB.

---

### Port conflicts

Another service is using one of the ports (8501, 8080, 8081, 9200, 9042, 5432, 6379, 2181). Stop the conflicting service or change the port mapping in `docker-compose.yml`.

---

## 10. Architecture Overview

### End-to-End Data Flow

```

Data Sources (TomTom, Open-Meteo, Simulated GPS)
  |
  v
Kafka Topics (traffic_events, weather_events, vehicle_position_events)
  |
  v
Spark Structured Streaming (3 jobs)
  |
  +---> Cassandra (congestion_metrics, weather_metrics, vehicle_speed_metrics)
  +---> Elasticsearch (congestion_alerts) ---> Streamlit Dashboard
  +---> Redis Pub/Sub (nileflow:alerts) ---> Discord Alerter

Airflow: daily corridor refresh, baseline calculation, data quality checks

```

### All 18 Docker Services

| #  | Service            | Port        | Purpose                           |
|----|--------------------|-------------|-----------------------------------|
| 1  | zookeeper          | 2181        | Kafka coordination                |
| 2  | kafka              | 9092, 29092 | Event streaming                   |
| 3  | postgres           | 5432        | Reference data + Airflow metadata |
| 4  | cassandra          | 9042        | Time-series metrics               |
| 5  | redis              | 6379        | Alert pub/sub + position cache    |
| 6  | elasticsearch      | 9200        | Alert and event log search        |
| 7  | cassandra-init     | -           | Auto-creates keyspace + tables    |
| 8  | elasticsearch-init | -           | Auto-creates indices              |
| 9  | airflow-init       | -           | DB migration + admin user         |
| 10 | airflow-webserver  | 8080        | DAG management UI                 |
| 11 | airflow-scheduler  | -           | DAG execution engine              |
| 12 | spark-master       | 7077, 8081  | Spark cluster manager             |
| 13 | spark-worker       | -           | Spark execution (6 cores, 3 GB)   |
| 14 | traffic-producer   | -           | TomTom API polling                |
| 15 | weather-producer   | -           | Open-Meteo API polling            |

|    |                            |      |                         |
|----|----------------------------|------|-------------------------|
| 16 | vehicle-positions-producer | -    | Simulated GPS pings     |
| 17 | dashboard                  | 8501 | Streamlit visualization |
| 18 | discord-alerter            | -    | Real-time alert posting |